

Міністерство освіти і науки України Національний технічний університет
України «Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №6
з дисципліни програмування комп'ютерної графіки
«Геометричне моделювання та анімація
рухомих об'єктів»

Виконав
Студент групи ІМ-24
Перчук Віталій Олександрович

Київ 2026

Мета роботи: отримати навички програмування відображення рухомих об'єктів тривимірних сцен.

Завдання.

Потрібно створити у середовищі Android Studio проєкт з ім'ям

Lab6_GLES, зокрема

- написати вихідний текст програми згідно варіанту завдання.

Використати мову Java або Kotlin – на вибір

- Налаштувати програму. Перевірити роботу програми на емуляторі та на фізичному пристрої Android.

Код:

MainActivity:

```
package com.example.lab6;
import android.content.Context;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import android.opengl.GLSurfaceView;
import android.opengl.GLES32;
import android.view.Menu;
import android.view.MenuItem;
import android.view.MotionEvent;
import android.view.ScaleGestureDetector;
import javax.microedition.khronos.egl.EGLConfig;
import javax.microedition.khronos.opengles.GL10;
public class MainActivity extends AppCompatActivity {
    private GLSurfaceView glView;
    private myWorkMode wmRef = null;
    private Context contextMain;
    private int currTab = 0;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        contextMain = this;
        glView = new MyGLSurfaceView(this);
        setContentView(glView);
    }
    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        menu.add(0, 1, 0, "Robot");
        menu.add(0, 2, 0, "Fish");
        return true;
    }
    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
        setTitle(item.getTitle());
        switch (item.getItemId()) {
            case 1:
                myModeStart(new robot(),
                    GLSurfaceView.RENDERMODE_CONTINUOUSLY);
                currTab = 1;
                return true;
            case 2:
                myModeStart(new fish(),
                    GLSurfaceView.RENDERMODE_CONTINUOUSLY);
                currTab = 2;
                return true;
            default:break;
        }
        return super.onOptionsItemSelected(item);
    }
    public void myModeStart(myWorkMode wmode, int rendermode) {
        wmRef = wmode;
    }
}
```

```

        glView.setRenderMode(rendermode);
        glView.requestRender();
    }
}

public class MyGLSurfaceView extends GLSurfaceView {
    private ScaleGestureDetector scaleDetector;
    private float scaleFactor = 1.0f;
    public MyGLSurfaceView(Context context){
        super(context);
        setEGLContextClientVersion(3);
        setRenderer(new MyGLRenderer());
        setRenderMode(GLSurfaceView.RENDERMODE_WHEN_DIRTY);
        scaleDetector = new ScaleGestureDetector(context, new ScaleListener());
    }
    @Override
    public boolean onTouchEvent(MotionEvent e) {
        if (wmRef == null) return false;
        if (wmRef.onTouchNotUsed()) return false;
        scaleDetector.onTouchEvent(e);
        int cx = this.getWidth();
        int cy = this.getHeight();
        float xtouch = e.getX();
        float ytouch = e.getY();
        switch (e.getActionMasked()) {
            case MotionEvent.ACTION_DOWN:
                if(wmRef.onActionDown(xtouch, ytouch, cx, cy)){
                    requestRender();
                }
                return true;
            case MotionEvent.ACTION_MOVE:
                if (!scaleDetector.isInProgress()) {
                    if(wmRef.onActionMove(xtouch, ytouch, cx, cy)){
                        requestRender();
                    }
                }
                return true;
        }
        return super.onTouchEvent(e);
    }
    private class ScaleListener extends ScaleGestureDetector.SimpleOnScaleGestureListener
    {
        @Override
        public boolean onScale(ScaleGestureDetector detector) {
            scaleFactor = detector.getScaleFactor();
            scaleFactor = Math.max(0.001f, Math.min(scaleFactor, 5.0f));
            if (wmRef != null) {
                if(wmRef.onScale(scaleFactor)){
                    requestRender();
                }
            }
            return true;
        }
    }
}

public class MyGLRenderer implements GLSurfaceView.Renderer {
    private int myRenderHeight = 1, myRenderWidth = 1;
    public void onSurfaceCreated(GL10 unused, EGLConfig config) {
        GLES32.glEnable(GLES32.GL_DEPTH_TEST);
        GLES32.glClearColor(0.8f, 0.85f, 0.9f, 1.0f);
    }
    public void onSurfaceChanged(GL10 unused, int width, int height) {
        GLES32.glViewport(0, 0, width, height);
        myRenderWidth = width;
        myRenderHeight = height;
    }
    public void onDrawFrame(GL10 unused) {
        GLES32.glClear(GLES32.GL_COLOR_BUFFER_BIT | GLES32.GL_DEPTH_BUFFER_BIT);
        if (wmRef == null) return;
        switch (currTab){
            case 1:
                wmRef.setCameraMat();
                wmRef.setLightMat();
                wmRef.setTexture(contextMain);
                if(!wmRef.myCreateShaderProgram(0)){
                    return;
                }
                wmRef.myCreateScene(0);
        }
    }
}

```



```

    }
    public int getProgramId() {
        return gl_Program;
    }
    protected int myCompileShader(int shadertype, String shadercode) {
        int shader_id = GLES32.glCreateShader(shadertype);
        GLES32.glShaderSource(shader_id, shadercode);
        GLES32.glCompileShader(shader_id);
        int[] res = new int[1];
        GLES32.glGetShaderiv(shader_id, GL_COMPILE_STATUS, res, 0);
        if (res[0] != 1) return 0;
        return shader_id;
    }
    protected boolean myCompileAndAttachShaders(String vsh, String fsh) {
        gl_Program = -1;
        int vertex_shader_id = myCompileShader(GLES32.GL_VERTEX_SHADER, vsh);
        if (vertex_shader_id == 0) return false;
        int fragment_shader_id = myCompileShader(GLES32.GL_FRAGMENT_SHADER, fsh);
        if (fragment_shader_id == 0) return false;
        gl_Program = GLES32.glCreateProgram();
        GLES32.glAttachShader(gl_Program, vertex_shader_id);
        GLES32.glAttachShader(gl_Program, fragment_shader_id);
        GLES32.glLinkProgram(gl_Program);
        GLES32.glDeleteShader(vertex_shader_id);
        GLES32.glDeleteShader(fragment_shader_id);
        if (getProgramId() == 0) return false;
        GLES32.glUseProgram(getProgramId());
        return true;
    }
    protected void getId_VAO_VBO() {
        int[] tmp = new int[2];
        GLES32.glGenVertexArrays(1, tmp, 0);
        VAO_id = tmp[0];
        GLES32.glGenBuffers(1, tmp, 0);
        VBO_id = tmp[0];
    }
    protected void myVertexArrayBind1(float[] src, int stride,
                                     String attrib1, int offset1)
    ) {
        if (gl_Program <= 0) return;
        ByteBuffer bb = ByteBuffer.allocateDirect(src.length*4);
        bb.order(ByteOrder.nativeOrder());
        FloatBuffer vertexBuffer = bb.asFloatBuffer();
        vertexBuffer.put(src);
        vertexBuffer.position(0);
        getId_VAO_VBO();
        GLES32.glBindVertexArray(VAO_id);
        GLES32.glBindBuffer(GLES32.GL_ARRAY_BUFFER, VBO_id);
        GLES32.glBufferData(GLES32.GL_ARRAY_BUFFER, src.length*4, vertexBuffer,
GLES32.GL_STATIC_DRAW);
        int handle = GLES32.glGetAttribLocation(gl_Program, attrib1);
        GLES32.glEnableVertexAttribArray(handle);
        GLES32.glVertexAttribPointer(handle, 3, GLES32.GL_FLOAT, false, stride*4, offset1);
        GLES32.glBindBuffer(GLES32.GL_ARRAY_BUFFER, 0);
        GLES32.glBindVertexArray(0);
    }
    protected void myVertexArrayBind2(float[] src, int stride,
                                     String attrib1, int offset1,
                                     String attrib2, int offset2)
    ) {
        if (gl_Program <= 0) return;
        ByteBuffer bb = ByteBuffer.allocateDirect(src.length*4);
        bb.order(ByteOrder.nativeOrder());
        FloatBuffer vertexBuffer = bb.asFloatBuffer();
        vertexBuffer.put(src);
        vertexBuffer.position(0);
        getId_VAO_VBO();
        GLES32.glBindVertexArray(VAO_id);
        GLES32.glBindBuffer(GLES32.GL_ARRAY_BUFFER, VBO_id);
        GLES32.glBufferData(GLES32.GL_ARRAY_BUFFER, src.length*4, vertexBuffer,
GLES32.GL_STATIC_DRAW);
        int handle = GLES32.glGetAttribLocation(gl_Program, attrib1);
        GLES32.glEnableVertexAttribArray(handle);
        GLES32.glVertexAttribPointer(handle, 3, GLES32.GL_FLOAT, false, stride*4, offset1);
        handle = GLES32.glGetAttribLocation(gl_Program, attrib2);
        GLES32.glEnableVertexAttribArray(handle);
    }

```

```

        GLES32.glVertexAttribPointer(handle, 3, GLES32.GL_FLOAT, false, stride*4, offset2);
        GLES32.glBindBuffer(GLES32.GL_ARRAY_BUFFER, 0);
        GLES32.glBindVertexArray(0);
    }
    protected void myVertexArrayBind3(float[] src, int stride,
                                     String atrib1, int offset1,
                                     String atrib2, int offset2,
                                     String atrib3, int offset3
    ) {
        if (gl Program <= 0) return;
        ByteBuffer bb = ByteBuffer.allocateDirect(src.length*4);
        bb.order(ByteOrder.nativeOrder());
        FloatBuffer vertexBuffer = bb.asFloatBuffer();
        vertexBuffer.put(src);
        vertexBuffer.position(0);
        getId VAO VBO();
        GLES32.glBindVertexArray(VAO_id);
        GLES32.glBindBuffer(GLES32.GL_ARRAY_BUFFER, VBO_id);
        GLES32.glBufferData(GLES32.GL_ARRAY_BUFFER, src.length*4, vertexBuffer,
        GLES32.GL_STATIC_DRAW);
        int handle;
        if (!atrib1.isEmpty()) {
            handle = GLES32.glGetAttribLocation(gl_Program, atrib1);
            GLES32.glEnableVertexAttribArray(handle);
            GLES32.glVertexAttribPointer(handle, 3, GLES32.GL_FLOAT, false, stride*4,
offset1);
        }
        if (!atrib2.isEmpty()) {
            handle = GLES32.glGetAttribLocation(gl_Program, atrib2);
            GLES32.glEnableVertexAttribArray(handle);
            GLES32.glVertexAttribPointer(handle, 3, GLES32.GL_FLOAT, false, stride*4,
offset2);
        }
        if (!atrib3.isEmpty()) {
            handle = GLES32.glGetAttribLocation(gl_Program, atrib3);
            GLES32.glEnableVertexAttribArray(handle);
            GLES32.glVertexAttribPointer(handle, 2, GLES32.GL_FLOAT, false, stride*4, offset3);
        }
        GLES32.glBindBuffer(GLES32.GL_ARRAY_BUFFER, 0);
        GLES32.glBindVertexArray(0);
    }
    protected void myVertexArrayBind4(float[] src, int stride,
                                     String atrib1, int offset1,
                                     String atrib2, int offset2,
                                     String atrib3, int offset3,
                                     String atrib4, int offset4,
                                     String atrib5, int offset5
    ) {
        if (gl Program <= 0) return;
        ByteBuffer bb = ByteBuffer.allocateDirect(src.length*4);
        bb.order(ByteOrder.nativeOrder());
        FloatBuffer vertexBuffer = bb.asFloatBuffer();
        vertexBuffer.put(src);
        vertexBuffer.position(0);
        getId VAO VBO();
        GLES32.glBindVertexArray(VAO_id);
        GLES32.glBindBuffer(GLES32.GL_ARRAY_BUFFER, VBO_id);
        GLES32.glBufferData(GLES32.GL_ARRAY_BUFFER, src.length*4, vertexBuffer,
        GLES32.GL_STATIC_DRAW);
        int handle;
        if (!atrib1.isEmpty()) {
            handle = GLES32.glGetAttribLocation(gl_Program, atrib1);
            GLES32.glEnableVertexAttribArray(handle);
            GLES32.glVertexAttribPointer(handle, 3, GLES32.GL_FLOAT, false, stride*4,
offset1);
        }
        if (!atrib2.isEmpty()) {
            handle = GLES32.glGetAttribLocation(gl_Program, atrib2);
            GLES32.glEnableVertexAttribArray(handle);
            GLES32.glVertexAttribPointer(handle, 3, GLES32.GL_FLOAT, false, stride*4,
offset2);
        }
        if (!atrib3.isEmpty()) {
            handle = GLES32.glGetAttribLocation(gl_Program, atrib3);
            GLES32.glEnableVertexAttribArray(handle);
            GLES32.glVertexAttribPointer(handle, 2, GLES32.GL_FLOAT, false, stride*4, offset3);
        }
    }

```

```

    }
    if (!atrib4.isEmpty()) {
        handle = GLES32.glGetAttribLocation(gl_Program, atrib4);
        GLES32.glEnableVertexAttribArray(handle);
        GLES32.glVertexAttribPointer(handle, 4, GLES32.GL_FLOAT, false, stride*4, offset4);
    }
    if (!atrib5.isEmpty()) {
        handle = GLES32.glGetAttribLocation(gl_Program, atrib5);
        GLES32.glEnableVertexAttribArray(handle);
        GLES32.glVertexAttribPointer(handle, 4, GLES32.GL_FLOAT, false, stride*4, offset5);
    }
    GLES32.glBindBuffer(GLES32.GL_ARRAY_BUFFER, 0);
    GLES32.glBindVertexArray(0);
}

public int myLoadTexture(Context context, int resourceId, int wrap) {
    int[] tmp = new int[2];
    GLES32.glGenTextures(1, tmp, 0);
    int handle = tmp[0];
    BitmapFactory.Options options = new BitmapFactory.Options();
    options.inScaled = false;
    Bitmap bitmap = BitmapFactory.decodeResource(context.getResources(),
        resourceId, options);
    GLES32.glBindTexture(GLES32.GL_TEXTURE_2D, handle);
    GLES32.glTexParameteri(GLES32.GL_TEXTURE_2D,
        GLES32.GL_TEXTURE_WRAP_S, wrap);
    GLES32.glTexParameteri(GLES32.GL_TEXTURE_2D,
        GLES32.GL_TEXTURE_WRAP_T, wrap);
    GLES32.glTexParameteri(GLES32.GL_TEXTURE_2D,
        GLES32.GL_TEXTURE_MIN_FILTER, GLES32.GL_LINEAR);
    GLES32.glTexParameteri(GLES32.GL_TEXTURE_2D,
        GLES32.GL_TEXTURE_MAG_FILTER, GLES32.GL_LINEAR);
    GLUtils.texImage2D(GLES32.GL_TEXTURE_2D, 0, bitmap, 0);
    bitmap.recycle();
    GLES32.glBindTexture(GLES32.GL_TEXTURE_2D, 0);
    return handle;
}

public boolean myCreateShaderProgram(int curr) {return false; }
public void setCameraMat() {}
public void setLightMat() {}
public void setTesture(Context context){}
public void getObjMat(int currObj){}
public void setObjMat(int currObj){}
public void myBindObjBuff(int curr){}
protected void myCreateScene(int curr) { }
public void myUseProgramForDrawing(int width, int height) {
    GLES32.glBindVertexArray(VAO_id);
    Matrix.setIdentityM(modelMatrix, 0);
    Matrix.translateM(modelMatrix, 0, objMat[0], objMat[1], objMat[2]);
    Matrix.rotateM(modelMatrix, 0, objMat[4], objMat[3], 0, 0);
    Matrix.rotateM(modelMatrix, 0, objMat[6], 0, objMat[5], 0);
    Matrix.rotateM(modelMatrix, 0, objMat[8], 0, 0, objMat[7]);
    Matrix.scaleM(modelMatrix, 0, objMat[9], objMat[10], objMat[11]);
    Matrix.setLookAtM(viewMatrix, 0,
        cameraMat[0], cameraMat[1], cameraMat[2],
        cameraMat[3], cameraMat[4], cameraMat[5],
        0, 1, 0);

    float aspect = (float) width / height;
    Matrix.perspectiveM(projMatrix, 0, 90, aspect, 0.1f, 100.0f);
    int handle = GLES32.glGetUniformLocation(gl_Program, "uModelMatrix");
    GLES32.glUniformMatrix4fv(handle, 1, false, modelMatrix, 0);
    handle = GLES32.glGetUniformLocation(gl_Program, "uViewMatrix");
    GLES32.glUniformMatrix4fv(handle, 1, false, viewMatrix, 0);
    handle = GLES32.glGetUniformLocation(gl_Program, "uProjMatrix");
    GLES32.glUniformMatrix4fv(handle, 1, false, projMatrix, 0);
    handle = GLES32.glGetUniformLocation(gl_Program, "vColor");
    GLES32.glUniform3f(handle, objColor[0], objColor[1], objColor[2]);
    handle = GLES32.glGetUniformLocation(gl_Program, "vLightPos");
    GLES32.glUniform3f(handle, lightObj[0], lightObj[1], lightObj[2]);
    handle = GLES32.glGetUniformLocation(gl_Program, "vLightColor");
    GLES32.glUniform3f(handle, lightObj[3], lightObj[4], lightObj[5]);
    handle = GLES32.glGetUniformLocation(gl_Program, "lightK");
    GLES32.glUniform3f(handle, lightObj[6], lightObj[7], lightObj[8]);
    handle = GLES32.glGetUniformLocation(gl_Program, "distK");
    GLES32.glUniform3f(handle, distK[0], distK[1], distK[2]);
    handle = GLES32.glGetUniformLocation(gl_Program, "vEyePos");
    GLES32.glUniform3f(handle, cameraMat[0], cameraMat[1], cameraMat[2]);
}

```

```

        int samplerLoc = GLES32.glGetUniformLocation(gl_Program, "vTextureSample");
        GLES32.glActiveTexture(GLES32.GL_TEXTURE0);
        GLES32.glBindTexture(GLES32.GL_TEXTURE_2D, textureHandle);
        GLES32.glUniform1i(samplerLoc, 0);
        GLES32.glDrawArrays(GLES32.GL_TRIANGLES, 0, numVertex);
        GLES32.glBindTexture(GLES32.GL_TEXTURE_2D, 0);
        GLES32.glBindVertexArray(0);
    }
    public void myUseProgramForDrawing0(int width, int height) {
        GLES32.glBindVertexArray(VAO_id);
        Matrix.setIdentityM(modelMatrix, 0);
        Matrix.translateM(modelMatrix, 0, lightObj[0], lightObj[1], lightObj[2]);
        Matrix.rotateM(modelMatrix, 0, 0, 1, 0, 0);
        Matrix.rotateM(modelMatrix, 0, 0, 0, 1, 0);
        Matrix.rotateM(modelMatrix, 0, 0, 0, 0, 1);
        Matrix.scaleM(modelMatrix, 0, 1, 1, 1);
        Matrix.setLookAtM(viewMatrix, 0,
            cameraMat[0], cameraMat[1], cameraMat[2],
            cameraMat[3], cameraMat[4], cameraMat[5],
            0, 1, 0);
        float aspect = (float) width / height;
        Matrix.perspectiveM(projMatrix, 0, 90, aspect, 0.1f, 100.0f);
        int handle = GLES32.glGetUniformLocation(gl_Program, "uModelMatrix");
        GLES32.glUniformMatrix4fv(handle, 1, false, modelMatrix, 0);
        handle = GLES32.glGetUniformLocation(gl_Program, "uViewMatrix");
        GLES32.glUniformMatrix4fv(handle, 1, false, viewMatrix, 0);
        handle = GLES32.glGetUniformLocation(gl_Program, "uProjMatrix");
        GLES32.glUniformMatrix4fv(handle, 1, false, projMatrix, 0);
        handle = GLES32.glGetUniformLocation(gl_Program, "vColor");
        GLES32.glUniform3f(handle, lightObj[3], lightObj[4], lightObj[5]);
        GLES32.glDrawArrays(GLES32.GL_POINTS, 0, numVertex);
        GLES32.glBindVertexArray(0);
    }
    int boneCount = 10;
    float[] boneMatrix = new float[boneCount * 16];
    public void myUseProgramForDrawing4(int width, int height) {
        GLES32.glBindVertexArray(VAO_id);
        Matrix.setIdentityM(modelMatrix, 0);
        Matrix.translateM(modelMatrix, 0, objMat[0], objMat[1], objMat[2]);
        Matrix.rotateM(modelMatrix, 0, objMat[4], objMat[3], 0, 0);
        Matrix.rotateM(modelMatrix, 0, objMat[6], 0, objMat[5], 0);
        Matrix.rotateM(modelMatrix, 0, objMat[8], 0, 0, objMat[7]);
        Matrix.scaleM(modelMatrix, 0, objMat[9], objMat[10], objMat[11]);
        Matrix.setLookAtM(viewMatrix, 0,
            cameraMat[0], cameraMat[1], cameraMat[2],
            cameraMat[3], cameraMat[4], cameraMat[5],
            0, 1, 0);
        float aspect = (float) width / height;
        Matrix.perspectiveM(projMatrix, 0, 90, aspect, 0.1f, 100.0f);
        int handle = GLES32.glGetUniformLocation(gl_Program, "uModelMatrix");
        GLES32.glUniformMatrix4fv(handle, 1, false, modelMatrix, 0);
        handle = GLES32.glGetUniformLocation(gl_Program, "uViewMatrix");
        GLES32.glUniformMatrix4fv(handle, 1, false, viewMatrix, 0);
        handle = GLES32.glGetUniformLocation(gl_Program, "uProjMatrix");
        GLES32.glUniformMatrix4fv(handle, 1, false, projMatrix, 0);
        handle = GLES32.glGetUniformLocation(gl_Program, "vColor");
        GLES32.glUniform3f(handle, objColor[0], objColor[1], objColor[2]);
        handle = GLES32.glGetUniformLocation(gl_Program, "vLightPos");
        GLES32.glUniform3f(handle, lightObj[0], lightObj[1], lightObj[2]);
        handle = GLES32.glGetUniformLocation(gl_Program, "vLightColor");
        GLES32.glUniform3f(handle, lightObj[3], lightObj[4], lightObj[5]);
        handle = GLES32.glGetUniformLocation(gl_Program, "lightK");
        GLES32.glUniform3f(handle, lightObj[6], lightObj[7], lightObj[8]);
        handle = GLES32.glGetUniformLocation(gl_Program, "distK");
        GLES32.glUniform3f(handle, distK[0], distK[1], distK[2]);
        handle = GLES32.glGetUniformLocation(gl_Program, "vEyePos");
        GLES32.glUniform3f(handle, cameraMat[0], cameraMat[1], cameraMat[2]);
        handle = GLES32.glGetUniformLocation(gl_Program, "uBonesMatrices");
        GLES32.glUniformMatrix4fv(handle, boneCount, false, boneMatrix, 0);
        int samplerLoc = GLES32.glGetUniformLocation(gl_Program, "vTextureSample");
        GLES32.glActiveTexture(GLES32.GL_TEXTURE0);
        GLES32.glBindTexture(GLES32.GL_TEXTURE_2D, textureHandle);
        GLES32.glUniform1i(samplerLoc, 0);
        GLES32.glDrawArrays(GLES32.GL_TRIANGLES, 0, numVertex);
        GLES32.glBindTexture(GLES32.GL_TEXTURE_2D, 0);
        GLES32.glBindVertexArray(0);
    }

```



```

    }
    public boolean onTouchNotUsed() { return true; }
    public boolean onActionDown(float x, float y, int cx, int cy) { return false; }
    public boolean onActionMove(float x, float y, int cx, int cy) { return false; }
    public boolean onScale(float scaleFactor){ return false;}
    public boolean animation(){return false;}
    public static int addVertexXYZn(float[] vdest, int pos,
                                   float x, float y, float z,
                                   float xn, float yn, float zn,
                                   float uvx, float uvy
    ){
        float N = (float)Math.sqrt(xn*xn+yn*yn+zn*zn);
        if (N <= 0) return pos;
        vdest[pos++] = x;
        vdest[pos++] = y;
        vdest[pos++] = z;
        vdest[pos++] = xn/N;
        vdest[pos++] = yn/N;
        vdest[pos++] = zn/N;
        vdest[pos++] = uvx;
        vdest[pos++] = uvy;
        return pos;
    };
    public static int addVertexXYZnBone(float[] vdest, int pos,
                                       float x, float y, float z,
                                       float xn, float yn, float zn,
                                       float uvx, float uvy,
                                       float[] boneID, float[] weight
    ){
        float N = (float)Math.sqrt(xn*xn+yn*yn+zn*zn);
        if (N <= 0) return pos;
        vdest[pos++] = x;
        vdest[pos++] = y;
        vdest[pos++] = z;
        vdest[pos++] = xn/N;
        vdest[pos++] = yn/N;
        vdest[pos++] = zn/N;
        vdest[pos++] = uvx;
        vdest[pos++] = uvy;
        vdest[pos++] = boneID[0];
        vdest[pos++] = boneID[1];
        vdest[pos++] = boneID[2];
        vdest[pos++] = boneID[3];
        vdest[pos++] = weight[0];
        vdest[pos++] = weight[1];
        vdest[pos++] = weight[2];
        vdest[pos++] = weight[3];
        return pos;
    };
    public static int addTriangleXYZn(float[] vdest, int pos,
                                     float x1, float y1, float z1,
                                     float x2, float y2, float z2,
                                     float x3, float y3, float z3,
                                     float uvx1, float uvy1,
                                     float uvx2, float uvy2,
                                     float uvx3, float uvy3
    ){
        float xn = (z2-z1)*(y3-y1) - (y2-y1)*(z3-z1);
        float yn = (x2-x1)*(z3-z1) - (z2-z1)*(x3-x1);
        float zn = (y2-y1)*(x3-x1) - (x2-x1)*(y3-y1);
        pos = addVertexXYZn(vdest, pos, x1, y1, z1, xn, yn, zn, uvx1, uvy1);
        pos = addVertexXYZn(vdest, pos, x2, y2, z2, xn, yn, zn, uvx2, uvy2);
        pos = addVertexXYZn(vdest, pos, x3, y3, z3, xn, yn, zn, uvx3, uvy3);
        return pos;
    }
    public static int addTriangleXYZnBone(float[] vdest, int pos,
                                       float x1, float y1, float z1,
                                       float x2, float y2, float z2,
                                       float x3, float y3, float z3,
                                       float uvx1, float uvy1,
                                       float uvx2, float uvy2,
                                       float uvx3, float uvy3,
                                       float[] boneID1, float[] weight1,
                                       float[] boneID2, float[] weight2,
                                       float[] boneID3, float[] weight3
    ){

```

```

        float xn = (z2-z1)*(y3-y1) - (y2-y1)*(z3-z1);
        float yn = (x2-x1)*(z3-z1) - (z2-z1)*(x3-x1);
        float zn = (y2-y1)*(x3-x1) - (x2-x1)*(y3-y1);
        pos = addVertexXYZnBone(vdest, pos, x1, y1, z1, xn, yn, zn, uvx1, uvy1, boneID1,
weight1);
        pos = addVertexXYZnBone(vdest, pos, x2, y2, z2, xn, yn, zn, uvx2, uvy2, boneID2,
weight2);
        pos = addVertexXYZnBone(vdest, pos, x3, y3, z3, xn, yn, zn, uvx3, uvy3, boneID3,
weight3);
        return pos;
    }
    public static int addRect(float[] vdest, int pos,
        float x1, float y1, float z1,
        float x2, float y2, float z2,
        float x3, float y3, float z3,
        float x4, float y4, float z4,
        float uvx1, float uvy1,
        float uvx2, float uvy2,
        float uvx3, float uvy3,
        float uvx4, float uvy4
    ){
        pos = addTriangleXYZn(
            vdest, pos,
            x1, y1, z1,
            x2, y2, z2,
            x3, y3, z3,
            uvx1, uvy1,
            uvx2, uvy2,
            uvx3, uvy3
        );
        pos = addTriangleXYZn(
            vdest, pos,
            x1, y1, z1,
            x3, y3, z3,
            x4, y4, z4,
            uvx1, uvy1,
            uvx3, uvy3,
            uvx4, uvy4
        );
        return pos;
    }
    public static int addRectBone(float[] vdest, int pos,
        float x1, float y1, float z1,
        float x2, float y2, float z2,
        float x3, float y3, float z3,
        float x4, float y4, float z4,
        float uvx1, float uvy1,
        float uvx2, float uvy2,
        float uvx3, float uvy3,
        float uvx4, float uvy4,
        float[] boneID1, float[] weight1,
        float[] boneID2, float[] weight2,
        float[] boneID3, float[] weight3,
        float[] boneID4, float[] weight4
    ){
        pos = addTriangleXYZnBone(
            vdest, pos,
            x1, y1, z1,
            x2, y2, z2,
            x3, y3, z3,
            uvx1, uvy1,
            uvx2, uvy2,
            uvx3, uvy3,
            boneID1, weight1,
            boneID2, weight2,
            boneID3, weight3
        );
        pos = addTriangleXYZnBone(
            vdest, pos,
            x1, y1, z1,
            x3, y3, z3,
            x4, y4, z4,
            uvx1, uvy1,
            uvx3, uvy3,
            uvx4, uvy4,
            boneID1, weight1,

```

```

        boneID3, weight3,
        boneID4, weight4
    );
    return pos;
}

public static int addCube(float[] vdest, int pos,
    float x1, float y1, float z1,
    float x2, float y2, float z2,
    float x3, float y3, float z3,
    float x4, float y4, float z4,
    float x5, float y5, float z5,
    float x6, float y6, float z6,
    float x7, float y7, float z7,
    float x8, float y8, float z8
){
    pos = addRect(
        vdest, pos,
        x8, y8, z8,
        x7, y7, z7,
        x3, y3, z3,
        x4, y4, z4,
        0.01f, 0.99f,
        0.99f, 0.99f,
        0.99f, 0.01f,
        0.01f, 0.01f
    );
    pos = addRect(
        vdest, pos,
        x7, y7, z7,
        x6, y6, z6,
        x2, y2, z2,
        x3, y3, z3,
        0.01f, 0.99f,
        0.99f, 0.99f,
        0.99f, 0.01f,
        0.01f, 0.01f
    );
    pos = addRect(
        vdest, pos,
        x6, y6, z6,
        x5, y5, z5,
        x1, y1, z1,
        x2, y2, z2,
        0.01f, 0.99f,
        0.99f, 0.99f,
        0.99f, 0.01f,
        0.01f, 0.01f
    );
    pos = addRect(
        vdest, pos,
        x5, y5, z5,
        x8, y8, z8,
        x4, y4, z4,
        x1, y1, z1,
        0.01f, 0.99f,
        0.99f, 0.99f,
        0.99f, 0.01f,
        0.01f, 0.01f
    );
    pos = addRect(
        vdest, pos,
        x5, y5, z5,
        x6, y6, z6,
        x7, y7, z7,
        x8, y8, z8,
        0.01f, 0.99f,
        0.99f, 0.99f,
        0.99f, 0.01f,
        0.01f, 0.01f
    );
    pos = addRect(
        vdest, pos,
        x4, y4, z4,
        x3, y3, z3,
        x2, y2, z2,
        x1, y1, z1,

```

```

        0.01f, 0.99f,
        0.99f, 0.99f,
        0.99f, 0.01f,
        0.01f, 0.01f
    );
    return pos;
}

public static class Point{
    public Point(float x, float y, float z){
        point[0] = x;
        point[1] = y;
        point[2] = z;
    }
    float[] point = new float[3];
    float[] boneID = new float[]{-1.0f,-1.0f,-1.0f,-1.0f};
    float[] weight = new float[]{-1.0f,-1.0f,-1.0f,-1.0f};
}

public static class mySkeletonArray {
    private float[] nodesXYZ;
    private int nodesNum;
    public float X, Y, Z;
    public float[] Vec4 = {0,0,0,1};
    public mySkeletonArray(int n) {
        nodesNum = n;
        nodesXYZ = new float[nodesNum*3];
    }
    public void setSkeletonPoint(int indx, float x, float y, float z) {
        if ((indx < 0)|| (indx >= nodesNum)) return;
        indx *= 3;
        nodesXYZ[indx++] = x;
        nodesXYZ[indx++] = y;
        nodesXYZ[indx++] = z;
    }
    public void getSkeletonPoint(int indx) {
        if ((indx < 0)|| (indx >= nodesNum)) return;
        indx *= 3;
        Vec4[0] = X = nodesXYZ[indx++];
        Vec4[1] = Y = nodesXYZ[indx++];
        Vec4[2] = Z = nodesXYZ[indx++];
        Vec4[3] = 1;
    }
}

public static int addCylinder(float[] vdest, int pos, int n, int m, float length, float r,
mySkeletonArray msa){
    Point[][] points = new Point[n][m+1];
    float stepX = length / (n - 1);
    float angleStep = (float) (2.0 * Math.PI / m);
    float x = -stepX * (n-1) / 2;
    for (int i = 0; i < n; i++) {
        x += stepX;
        for (int j = 0; j < m; j++) {
            float angle = angleStep * j;
            float y = (float) (r * Math.cos(angle));
            float z = (float) (r * Math.sin(angle));
            points[i][j] = new Point(x, y, z);
        }
    }
    for(int i = 0; i < n; i++){
        Point t = points[i][0];
        points[i][m] = new Point(t.point[0], t.point[1], t.point[2]);
    }
    msa.setSkeletonPoint(0, points[0][0].point[0],0.0f, 0.0f);
    msa.setSkeletonPoint(1, points[10][0].point[0],0.0f, 0.0f);
    msa.setSkeletonPoint(2, points[14][0].point[0],0.0f, 0.0f);
    msa.setSkeletonPoint(3, points[16][0].point[0],0.0f, 0.0f);
    //fish модифікація початок
    float zK1 = 0.5f / 7;
    for(int i = 0; i < 8; i++){
        float k = (float)i / 7.0f;
        float scale = (float)Math.sin(k * Math.PI / 2.0);
        for(int j = 0; j < m+1; j++){
            Point t = points[i][j];
            points[i][j].point[1] = t.point[1] * scale;
            points[i][j].point[2] = t.point[2] * i * zK1;
        }
    }
}

```

```

float zK2 = -0.35f / 7;
float zKs2 = 0.5f;
for(int i = 15; i > 7; i--){
    float k = (float)(i) / 7.0f;
    float p = (float)Math.sin(k * Math.PI / 2.0);
    float scale = 0.5f + 0.5f * p;
    for(int j = 0; j < m+1; j++){
        Point t = points[i][j];
        points[i][j].point[1] = t.point[1] * scale;
        points[i][j].point[2] = t.point[2] * (zKs2 + (i-8) * zK2);
    }
}
float zK3 = 0.10f / 2;
float zKs3 = 0.15f;
for(int i = 16; i < 18; i++){
    float k = (float)(i - 16) / 2.0f;
    float p = (float)Math.sin(k * Math.PI / 2.0);
    float scale = 0.5f + 0.20f * p;
    for(int j = 0; j < m+1; j++){
        Point t = points[i][j];
        points[i][j].point[1] = t.point[1] * scale;
        points[i][j].point[2] = t.point[2] * (zKs3 + (i-15) * zK3);
    }
}
float zK4 = -0.25f / 3;
float zKs4 = 0.25f;
for(int i = 20; i > 17; i--) {
    float k = (float)(i) / 3.0f;
    float p = (float) Math.sin(k * Math.PI / 2.0);
    float scale = 0.7f + 0.075f * p;
    for (int j = 0; j < m + 1; j++) {
        Point t = points[i][j];
        points[i][j].point[1] = t.point[1] * scale;
        points[i][j].point[2] = t.point[2] * (zKs4 + (i - 17) * zK4);
    }
}
for(int i = 17; i < 21; i++){
    for (int j = 2; j < 6; j++) {
        float k = (float)(j-2) / 4.0f;
        float p = (float)Math.sin(k * Math.PI / 2.0);
        float scale = 1.0f - 0.075f * p;
        Point t = points[i][j];
        points[i][j].point[0] = t.point[0] * scale;
    }
    for (int j = 8; j > 5; j--) {
        float k = (float)(j-8) / 3.0f;
        float p = (float)Math.sin(k * Math.PI / 2.0);
        float scale = 1.0f + 0.075f * p;
        Point t = points[i][j];
        points[i][j].point[0] = t.point[0] * scale;
    }
    for (int j = 13; j < 16; j++) {
        float k = (float)(j-13) / 3.0f;
        float p = (float)Math.sin(k * Math.PI / 2.0);
        float scale = 1.0f - 0.075f * p;
        Point t = points[i][j];
        points[i][j].point[0] = t.point[0] * scale;
    }
    for (int j = 19; j > 15; j--) {
        float k = (float)(j-19) / 4.0f;
        float p = (float)Math.sin(k * Math.PI / 2.0);
        float scale = 1.0f + 0.075f * p;
        Point t = points[i][j];
        points[i][j].point[0] = t.point[0] * scale;
    }
}
for(int i = 0; i < 6; i++){
    for(int j = 0; j < m+1; j++){
        points[i][j].boneID[0] = 0.0f;
        points[i][j].weight[0] = 1.0f;
    }
}
for (int j = 0; j < m + 1; j++) {
    points[6][j].boneID[0] = 0.0f;
    points[6][j].weight[0] = 0.95f;
    points[6][j].boneID[1] = 1.0f;
}

```

```

        points[6][j].weight[1] = 0.05f;
    }
    for(int i = 7; i < 10; i++) {
        for (int j = 0; j < m + 1; j++) {
            points[i][j].boneID[0] = 0.0f;
            points[i][j].weight[0] = 0.9f - 0.1f * (i-7);
            points[i][j].boneID[1] = 1.0f;
            points[i][j].weight[1] = 0.1f + 0.1f * (i-7);
        }
    }
    for(int i = 10; i < 15; i++) {
        for (int j = 0; j < m + 1; j++) {
            points[i][j].boneID[0] = 0.0f;
            points[i][j].weight[0] = 0.5f - 0.1f * (i-10);
            points[i][j].boneID[1] = 1.0f;
            points[i][j].weight[1] = 0.5f + 0.1f * (i-10);
        }
    }
    for (int j = 0; j < m + 1; j++) {
        points[15][j].boneID[0] = 1.0f;
        points[15][j].weight[0] = 0.3f;
        points[15][j].boneID[1] = 2.0f;
        points[15][j].weight[1] = 0.6f;
        points[15][j].boneID[2] = 3.0f;
        points[15][j].weight[2] = 0.1f;
    }
    for(int i = 16; i < 20; i++) {
        for (int j = 0; j < m + 1; j++) {
            points[i][j].boneID[0] = 2.0f;
            points[i][j].weight[0] = 0.5f - 0.1f * (i-16);
            points[i][j].boneID[1] = 3.0f;
            points[i][j].weight[1] = 0.5f + 0.1f * (i-16);
        }
    }
    for (int j = 0; j < m + 1; j++) {
        points[20][j].boneID[0] = 3.0f;
        points[20][j].weight[0] = 1.0f;
    }
    float textureStepX = 1.0f / (n - 1);
    float textureStepY = 1.0f / m;
    for(int i = 0; i < n - 1; i++){
        for(int j = 0; j < m; j++){
            Point t1 = points[i][j];
            Point t2 = points[i+1][j];
            Point t3 = points[i+1][j+1];
            Point t4 = points[i][j+1];
            float u0 = i * textureStepX;
            float u1 = (i + 1) * textureStepX;
            float v0 = j * textureStepY;
            float v1 = (j + 1) * textureStepY;
            pos = addRectBone(vdest, pos,
                t1.point[0], t1.point[1], t1.point[2],
                t2.point[0], t2.point[1], t2.point[2],
                t3.point[0], t3.point[1], t3.point[2],
                t4.point[0], t4.point[1], t4.point[2],
                u0, v0,
                u1, v0,
                u1, v1,
                u0, v1,
                t1.boneID, t1.weight,
                t2.boneID, t2.weight,
                t3.boneID, t3.weight,
                t4.boneID, t4.weight
            );
        }
    }
    Point t6 = points[6][0];
    Point t9 = points[9][0];
    Point t8 = points[8][0];
    float u0 = 0;
    float u1 = 1 * textureStepX/1000;
    float v0 = 0;
    float v1 = 1 * textureStepY/1000;

    pos = addRectBone(vdest, pos,
        t6.point[0], t6.point[1], t6.point[2],

```

```

        t8.point[0], t8.point[1]+0.05f, t8.point[2],
        t9.point[0], t9.point[1]+0.025f, t9.point[2],
        t8.point[0], t8.point[1], t8.point[2],
        u0, v0,
        u1, v0,
        u1, v1,
        u0, v1,
        t6.boneID, t6.weight,
        t8.boneID, t8.weight,
        t9.boneID, t9.weight,
        t8.boneID, t8.weight
    );
    Point t11 = points[11][0];
    pos = addRectBone(vdest, pos,
        t8.point[0], t8.point[1], t8.point[2],
        t9.point[0], t9.point[1]+0.025f, t9.point[2],
        t11.point[0]+stepX/2, t11.point[1]+0.01f, t11.point[2],
        t11.point[0], t11.point[1], t11.point[2],
        u0, v0,
        u1, v0,
        u1, v1,
        u0, v1,
        t8.boneID, t8.weight,
        t9.boneID, t9.weight,
        t11.boneID, t11.weight,
        t11.boneID, t11.weight
    );
    Point t12 = points[12][11];
    Point t13 = points[13][11];
    Point t14 = points[14][11];
    pos = addRectBone(vdest, pos,
        t12.point[0], t12.point[1], 0.0f,
        t13.point[0], t13.point[1], 0.0f,
        t14.point[0], t14.point[1]-0.02f, 0.0f,
        t14.point[0], t14.point[1]-0.05f, 0.0f,
        u0, v0,
        u1, v0,
        u1, v1,
        u0, v1,
        t12.boneID, t12.weight,
        t13.boneID, t13.weight,
        t14.boneID, t14.weight,
        t14.boneID, t14.weight
    );
    Point t4 = points[4][8];
    Point t6_8 = points[6][8];
    msa.setSkeletonPoint(4, t4.point[0], t4.point[1], t4.point[2]);
    pos = addRectBone(vdest, pos,
        t4.point[0], t4.point[1], t4.point[2],
        t4.point[0], t4.point[1], t4.point[2],
        t6_8.point[0]+stepX/2, t6_8.point[1]-0.02f, t6_8.point[2]+0.01f,
        t6_8.point[0]-stepX/2, t6_8.point[1]-0.05f, t6_8.point[2]+0.03f,
        u0, v0,
        u1, v0,
        u1, v1,
        u0, v1,
        new float[]{4.0f, 0.0f, 0.0f, 0.0f}, new float[]{1.0f, 0.0f, 0.0f, 0.0f},
        new float[]{4.0f, 0.0f, 0.0f, 0.0f}, new float[]{1.0f, 0.0f, 0.0f, 0.0f},
        new float[]{4.0f, 0.0f, 0.0f, 0.0f}, new float[]{1.0f, 0.0f, 0.0f, 0.0f},
        new float[]{4.0f, 0.0f, 0.0f, 0.0f}, new float[]{1.0f, 0.0f, 0.0f, 0.0f}
    );
    Point t4_14 = points[4][13];
    Point t6_14 = points[6][13];
    msa.setSkeletonPoint(5, t4_14.point[0], t4_14.point[1], t4_14.point[2]);
    pos = addRectBone(vdest, pos,
        t4_14.point[0], t4_14.point[1], t4_14.point[2],
        t4_14.point[0], t4_14.point[1], t4_14.point[2],
        t6_14.point[0]-stepX/2, t6_14.point[1]-0.05f, t6_14.point[2]-0.03f,
        t6_14.point[0]+stepX/2, t6_14.point[1]-0.02f, t6_14.point[2]-0.01f,
        u0, v0,
        u1, v0,
        u1, v1,
        u0, v1,
        new float[]{5.0f, 0.0f, 0.0f, 0.0f}, new float[]{1.0f, 0.0f, 0.0f, 0.0f},
        new float[]{5.0f, 0.0f, 0.0f, 0.0f}, new float[]{1.0f, 0.0f, 0.0f, 0.0f},
        new float[]{5.0f, 0.0f, 0.0f, 0.0f}, new float[]{1.0f, 0.0f, 0.0f, 0.0f},
        new float[]{5.0f, 0.0f, 0.0f, 0.0f}, new float[]{1.0f, 0.0f, 0.0f, 0.0f}
    );

```

```

        new float[]{5.0f, 0.0f, 0.0f, 0.0f}, new float[]{1.0f, 0.0f, 0.0f, 0.0f}
    );
    return pos;
}
}

```

myShaders:

```

package com.example.lab6;
public class myShaders {
    public static final String vertexShaderCode4 =
        "#version 300 es\n" +
        "uniform mat4 uModelMatrix;\n" +
        "uniform mat4 uViewMatrix;\n" +
        "uniform mat4 uProjMatrix;\n" +
        "in vec3 vPosition;\n" +
        "in vec3 vNormal;\n" +
        "in vec2 vTexture;\n" +
        "out vec3 currentPos;\n" +
        "out vec3 currentNormal;\n" +
        "out vec2 currentTextCoord;\n" +
        "void main() {\n" +
        "    gl_Position = uProjMatrix * uViewMatrix * uModelMatrix * vec4(vPosition,
1.0f);\n" +
        "    vec4 worldPos = uModelMatrix * vec4(vPosition, 1.0);\n" +
        "    currentPos = worldPos.xyz;\n" +
        "    mat3 normalMat = mat3(transpose(inverse(uModelMatrix)));\n" +
        "    currentNormal = normalize(normalMat * vNormal);\n" +
        "    currentTextCoord = vec2(vTexture.x, vTexture.y);\n" +
        "}\n";
    public static final String fragmentShaderCode4 =
        "#version 300 es\n" +
        "precision mediump float;\n" +
        "in vec3 currentPos;\n" +
        "in vec3 currentNormal;\n" +
        "in vec2 currentTextCoord;\n" +
        "uniform vec3 vColor;\n" +
        "uniform vec3 vLightColor;\n" +
        "uniform vec3 vLightPos;\n" +
        "uniform vec3 vEyePos;\n" +
        "uniform vec3 distK;\n" +
        "uniform vec3 lightK;\n" +
        "uniform sampler2D vTextureSample;\n" +
        "out vec4 resultColor;\n" +
        "void main() {\n" +
        "    vec3 norm = normalize(currentNormal);\n" +
        "    vec3 lightDir = normalize(vLightPos - currentPos);\n" +
        "    float distance = length(vLightPos - currentPos);\n" +
        "    float attenuation = 1.0f / (1.0f + distK[0] * distance * distance);\n" +
        "    float diffuse = max(dot(norm, lightDir), 0.0);\n" +
        "    vec3 reflectDir = normalize(reflect(-lightDir, norm));\n" +
        "    vec3 eyeDir = normalize(vEyePos - currentPos);\n" +
        "    float spec = max(dot(eyeDir, reflectDir), 0.0f);\n" +
        "    spec = pow(spec, 200.0f);\n" +
        "    vec3 vClr = lightK[0] * vColor + lightK[1] * diffuse * vLightColor *
vColor + lightK[2] * spec * vLightColor;\n" +
        "    resultColor = texture(vTextureSample, currentTextCoord) *
vec4(attenuation * vClr, 1.0f);\n" +
        "}\n";
    public static final String vertexShaderCode5 =
        "#version 300 es\n" +
        "uniform mat4 uModelMatrix;\n" +
        "uniform mat4 uViewMatrix;\n" +
        "uniform mat4 uProjMatrix;\n" +
        "in vec3 vPosition;\n" +
        "void main() {\n" +
        "    gl_Position = uProjMatrix * uViewMatrix * uModelMatrix * vec4(vPosition,
1.0f);\n" +
        "    gl_PointSize = 15.0f;\n" +
        "}\n";
    public static final String fragmentShaderCode5 =

```



```

        "#version 300 es\n" +
        "precision mediump float;\n" +
        "uniform vec3 vColor;\n" +
        "out vec4 outColor;\n" +
        "void main() {\n" +
        "    outColor = vec4(vColor, 1.0f);\n" +
        "}\n";
public static final String vertexShaderCode6 =
    "#version 300 es\n" +
    "uniform mat4 uModelMatrix;\n" +
    "uniform mat4 uViewMatrix;\n" +
    "uniform mat4 uProjMatrix;\n" +
    "const int MAX_BONES = 10;\n" +
    "const int MAX_BONE_INFLUENCE = 4;\n" +
    "uniform mat4 uBonesMatrices[MAX_BONES];\n" +
    "in vec3 vPosition;\n" +
    "in vec3 vNormal;\n" +
    "in vec2 vTexture;\n" +
    "in vec4 vBoneIDs;\n" +
    "in vec4 vWeights;\n" +
    "out vec3 currentPos;\n" +
    "out vec3 currentNormal;\n" +
    "out vec2 currentTextCoord;\n" +
    "void main() {\n" +
    "    vec4 tPosition = vec4(0.0);\n" +
    "    vec3 tNormal = vec3(0.0);\n" +
    "    for(int i=0; i < MAX_BONE_INFLUENCE; i++) {\n" +
    "        int boneID = int(vBoneIDs[i]);\n" +
    "        if ((boneID < 0) || (boneID >= MAX_BONES)) break;\n" +
    "        mat4 boneMatrix = uBonesMatrices[boneID];\n" +
    "        tPosition += vWeights[i] * (boneMatrix * vec4(vPosition, 1.0));\n" +
    "        tNormal += vWeights[i] * (mat3(boneMatrix) * vNormal);\n" +
    "    }\n" +
    "    if(tPosition == vec4(0.0)){\n" +
    "        tPosition = vec4(vPosition, 1.0);\n" +
    "        tNormal = vNormal;\n" +
    "    }\n" +
    "    gl_Position = uProjMatrix * uViewMatrix * uModelMatrix * tPosition;\n" +
    "    vec4 worldPos = uModelMatrix * tPosition;\n" +
    "    currentPos = worldPos.xyz;\n" +
    "    mat3 normalMat = mat3(transpose(inverse(uModelMatrix)));\n" +
    "    currentNormal = normalize(normalMat * tNormal);\n" +
    "    currentTextCoord = vec2(vTexture.x, vTexture.y);\n" +
    "    }\n";
}

```

Fish:

```

package com.example.lab6;
import static java.lang.Math.cos;
import static java.lang.Math.sin;
import android.content.Context;
import android.opengl.GLES32;
import android.opengl.Matrix;
public class fish extends myWorkMode{
    fish() {
        super();
    }
    float[] cameraMat1 = new float[]{
        0.0f, 3.0f, 5.0f,
        0.0f, 0.0f, 0.0f,
        0.0f,
        5.0f,
        0.0f
    };
    public float [] objMat1 = new float[]{
        0.0f, 0.5f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        4.0f,1.0f,4.0f
    };
    public float [] objMat2 = new float[]{
        0.0f, 0.0f, 0.0f,

```

```

        1.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        6.0f, 6.0f, 6.0f
    };
    float[] lightObj1 = new float[]{
        0.0f, 3.0f, 0.0f,
        1.0f, 1.0f, 1.0f,
        0.3f, 0.7f, 0.0f
    };
    @Override
    public void setCameraMat(){
        cameraMat = cameraMat1;
    }
    @Override
    public void setLightMat(){
        lightObj = lightObj1;
        distK = new float[]{
            0.0f, 1.0f, 1.0f
        };
    }
    private int textureHandle1 = 0;
    private int textureHandle2 = 0;
    private boolean textureLoad = false;
    @Override
    public void setTesture(Context context){
        if(!textureLoad) {
            textureHandle1 = myLoadTexture(context, R.drawable.cells, GLES32.GL_REPEAT);
            textureHandle2 = myLoadTexture(context, R.drawable.fish, GLES32.GL_REPEAT);
            textureLoad = true;
        }
    }
    private float[] arrVert1;
    private boolean vert1Init = false;
    private float[] arrVert2;
    private boolean vert2Init = false;
    private mySkeletonArray msa = new mySkeletonArray(6);
    @Override
    protected void myCreateScene(int curr) {
        switch (curr){
            case 0:
                arrayVertex = new float[]{
                    0.0f, 0.0f, 0.0f,
                };
                numVertex = 1;
                break;
            case 1:
                if(!vert1Init) {
                    arrVert1 = new float[6 * 8];
                    int pos = 0;
                    addRect(
                        arrVert1, pos,
                        -0.5f, -0.5f, -0.5f,
                        0.5f, -0.5f, -0.5f,
                        0.5f, -0.5f, 0.5f,
                        -0.5f, -0.5f, 0.5f,
                        0.01f, 3.99f,
                        3.99f, 3.99f,
                        3.99f, 0.01f,
                        0.01f, 0.01f
                    );
                    vert1Init = true;
                }
                objColor = new float[]{
                    1.0f, 1.0f, 1.0f
                };
                arrayVertex = arrVert1;
                numVertex = 6;
                textureHandle = textureHandle1;
                break;
            case 2:
                int n = 21;
                int m = 21;
                int rectCount = (n-1)*(m+1) + 5;
                float lenght = 0.5f;
                float r = 0.1f;

```

```

        if(!vert2Init) {
            arrVert2 = new float[rectCount * 6 * 16];
            int pos = 0;
            addCylinder(arrVert2, pos, n, m, lenght, r, msa);
            vert2Init = true;
        }
        objColor = new float[]{
            1.0f, 1.0f, 1.0f
        };
        arrayVertex = arrVert2;
        numVertex = rectCount * 6;
        textureHandle = textureHandle2;
        break;
    default:
        break;
    }
}

@Override
public void setObjMat(int currObj){
    if(currObj == 1){
        objMat1 = objMat.clone();
    }
    if(currObj == 2){
        objMat2 = objMat.clone();
    }
}

@Override
public void getObjMat(int currObj){
    if(currObj == 1){
        objMat = objMat1.clone();
    }
    if(currObj == 2){
        objMat = objMat2.clone();
    }
}

@Override
public boolean myCreateShaderProgram(int curr) {
    switch (curr){
        case 0:
            return myCompileAndAttachShaders(myShaders.vertexShaderCode5,
myShaders.fragmentShaderCode5);
        case 2:
            return myCompileAndAttachShaders(myShaders.vertexShaderCode4,
myShaders.fragmentShaderCode4);
        case 3:
            return myCompileAndAttachShaders(myShaders.vertexShaderCode6,
myShaders.fragmentShaderCode4);
        default:
            return false;
    }
}

@Override
public void myBindObjBuff(int curr) {
    switch (curr){
        case 0:
            myVertexArrayBind1(arrayVertex, 3,"vPosition", 0);
            break;
        case 2:
            myVertexArrayBind3(arrayVertex, 8,"vPosition", 0,"vNormal", 3*4, "vTexture",
6*4);
            break;
        case 3:
            myVertexArrayBind4(arrayVertex, 16,"vPosition", 0,"vNormal", 3*4, "vTexture",
6*4, "vBoneIDs", 8*4, "vWeights", 12*4);
            break;
        default:
            break;
    }
}

@Override
public boolean onTouchNotUsed() { return false; }

@Override
public boolean onActionDown(float x, float y, int cx, int cy){
    prevDownX = x;
    prevDownY = y;
    return true;
}

```

```

};
@Override
public boolean onActionMove(float x, float y, int cx, int cy){
    float dx = x - prevDownX;
    float dy = y - prevDownY;
    prevDownX = x;
    prevDownY = y;
    cameraMat[6] -= dx * sensitivity;
    cameraMat[8] += dy * sensitivity;
    float limit = 1.5f;
    if (cameraMat[8] > limit) cameraMat[8] = limit;
    if (cameraMat[8] < -limit) cameraMat[8] = -limit;
    updateCameraOrbitY();
    return true;
};
private void updateCameraOrbitY() {
    float cx = 0.0f;
    float cy = 0.0f;
    float cz = 0.0f;
    float yaw = cameraMat[6];
    float radius = cameraMat[7];
    float pitch = cameraMat[8];
    float x = cx + radius * (float)(cos(pitch) * sin(yaw));
    float y = cy + radius * (float)(sin(pitch));
    float z = cz + radius * (float)(cos(pitch) * cos(yaw));
    cameraMat[0] = x;
    cameraMat[1] = y;
    cameraMat[2] = z;
    cameraMat[3] = cx;
    cameraMat[4] = cy;
    cameraMat[5] = cz;
}
@Override
public boolean onScale(float scaleFactor){
    float centerScaleFactor = 1.0f;
    float moveSensitivity = 0.1f;
    if(centerScaleFactor > scaleFactor){
        cameraMat[7] -= moveSensitivity;
    }
    else {
        cameraMat[7] += moveSensitivity;
    }
    cameraMat[7] = Math.max(1.0f, Math.min(cameraMat[7], 10.0f));
    updateCameraOrbitY();
    return true;
}
private float angleXrebet = 0.0f;
private float angleStep = 0.3f;
@Override
public boolean animation(){
    for(int i = 0; i < boneCount; i++){
        Matrix.setIdentityM(boneMatrix, i*16);
    }
    if(angleXrebet >= 7.0f || angleXrebet <= -7.0f){
        angleStep = -angleStep;
    }
    angleXrebet += angleStep;
    float angle = angleXrebet;
    for (int bone=1; bone<4; bone++) {
        angle += angleXrebet;
        msa.getSkeletonPoint(bone);
        int offsetPrev = 16*(bone-1);
        Matrix.multiplyMV (msa.Vec4, 0,
            boneMatrix, offsetPrev,
            msa.Vec4, 0);
        int offsetCurrent = offsetPrev + 16;
        Matrix.translateM(boneMatrix, offsetCurrent,
            msa.Vec4[0], msa.Vec4[1], msa.Vec4[2]);
        Matrix.rotateM(boneMatrix, offsetCurrent, angle, 0, 1, 0);
        Matrix.translateM(boneMatrix, offsetCurrent,
            -msa.X, -msa.Y, -msa.Z);
    }
    int offsetPrev = 0;
    msa.getSkeletonPoint(4);
    Matrix.multiplyMV (msa.Vec4, 0,
        boneMatrix, offsetPrev,

```

```

        msa.Vec4, 0);
int offsetCurrent = 16*(4);
Matrix.translateM(boneMatrix, offsetCurrent,
    msa.Vec4[0], msa.Vec4[1], msa.Vec4[2]);
Matrix.rotateM(boneMatrix, offsetCurrent, -angle, 0, 1, 0);
Matrix.translateM(boneMatrix, offsetCurrent,
    -msa.X, -msa.Y, -msa.Z);
msa.getSkeletonPoint(5);
Matrix.multiplyMV (msa.Vec4, 0,
    boneMatrix, offsetPrev,
    msa.Vec4, 0);
offsetCurrent = 16*(5);
Matrix.translateM(boneMatrix, offsetCurrent,
    msa.Vec4[0], msa.Vec4[1], msa.Vec4[2]);
Matrix.rotateM(boneMatrix, offsetCurrent, angle, 0, 1, 0);
Matrix.translateM(boneMatrix, offsetCurrent,
    -msa.X, -msa.Y, -msa.Z);
return true;
}
}
}

```

Robot:

```

package com.example.lab6;
import static java.lang.Math.cos;
import static java.lang.Math.sin;
import java.lang.Math;
import android.content.Context;
import android.opengl.GLES32;
public class robot extends myWorkMode{
    robot() {
        super();
    }
    float[] cameraMat1 = new float[]{
        0.0f, 3.0f, 5.0f,
        0.0f, 0.0f, 0.0f,
        0.0f,
        5.0f,
        0.0f
    };
    public float [] objMat1 = new float[]{
        0.0f, 0.5f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        4.0f,1.0f,4.0f
    };
    public static class Part{
        public Part(float x, float y, float z,
            float rx, float rxAngle,
            float ry, float ryAngle,
            float rz, float rzAngle,
            float sx, float sy, float sz
        ){
            partMat[0] = x;
            partMat[1] = y;
            partMat[2] = z;
            partMat[3] = rx;
            partMat[4] = rxAngle;
            partMat[5] = ry;
            partMat[6] = ryAngle;
            partMat[7] = rz;
            partMat[8] = rzAngle;
            partMat[9] = sx;
            partMat[10] = sy;
            partMat[11] = sz;
        }
        public float [] partMat = new float[12];
    }
    public Part [] robot = new Part[]{
        // 0 head
        new Part(
            0.0f, 2.15f, -2.0f,
            1.0f, 0.0f,

```

```

        1.0f, 0.0f,
        1.0f, 0.0f,
        0.3f, 0.3f, 0.3f
    ),
    // 1 body
    new Part(
        0.0f, 1.6f, -2.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        0.8f, 0.8f, 0.4f
    ),
    // 2 права нога ч1
    new Part(
        -0.225f, 0.95f, -2.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        0.2f, 0.5f, 0.2f
    ),
    // 3 права нога ч2
    new Part(
        -0.225f, 0.45f, -2.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        0.2f, 0.5f, 0.2f
    ),
    // 4 права нога ч3
    new Part(
        -0.225f, 0.1f, -2.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        0.3f, 0.2f, 0.3f
    ),
    // 5 ліва нога ч1
    new Part(
        0.225f, 0.95f, -2.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        0.2f, 0.5f, 0.2f
    ),
    // 6 ліва нога ч2
    new Part(
        0.225f, 0.45f, -2.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        0.2f, 0.5f, 0.2f
    ),
    // 7 ліва нога ч3
    new Part(
        0.225f, 0.1f, -2.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        0.3f, 0.2f, 0.3f
    ),
    // 8 права рука ч1
    new Part(
        -0.5f, 1.75f, -2.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        0.2f, 0.5f, 0.2f
    ),
    // 9 права рука ч2
    new Part(
        -0.5f, 1.25f, -2.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        1.0f, 0.0f,
        0.2f, 0.5f, 0.2f
    ),
    ),

```

```

        // 10 права рука ч3
        new Part(
            -0.5f, 0.9f, -2.0f,
            1.0f, 0.0f,
            1.0f, 0.0f,
            1.0f, 0.0f,
            0.1f, 0.2f, 0.25f
        ),
        // 11 ліва рука ч1
        new Part(
            0.5f, 1.75f, -2.0f,
            1.0f, 0.0f,
            1.0f, 0.0f,
            1.0f, 0.0f,
            0.2f, 0.5f, 0.2f
        ),
        // 12 ліва рука ч2
        new Part(
            0.5f, 1.25f, -2.0f,
            1.0f, 0.0f,
            1.0f, 0.0f,
            1.0f, 0.0f,
            0.2f, 0.5f, 0.2f
        ),
        // 13 ліва рука ч3
        new Part(
            0.5f, 0.9f, -2.0f,
            1.0f, 0.0f,
            1.0f, 0.0f,
            1.0f, 0.0f,
            0.1f, 0.2f, 0.25f
        )
    };
    float[] lightObj1 = new float[]{
        0.0f, 3.0f, 0.0f,
        1.0f, 1.0f, 1.0f,
        0.3f, 0.7f, 0.0f
    };
    @Override
    public void setCameraMat(){
        cameraMat = cameraMat1;
    }
    @Override
    public void setLightMat(){
        lightObj = lightObj1;
        distK = new float[]{
            0.0f, 1.0f, 1.0f
        };
    }
    private int textureHandle1 = 0;
    private int textureHandle2 = 0;
    private boolean textureLoad = false;
    @Override
    public void setTesture(Context context){
        if(!textureLoad) {
            textureHandle1 = myLoadTexture(context, R.drawable.cells, GLES32.GL_REPEAT);
            textureHandle2 = myLoadTexture(context, R.drawable.robot, GLES32.GL_REPEAT);
            textureLoad = true;
        }
    }
    private float[] arrVert1;
    private boolean vert1Init = false;
    private float[] arrVert2;
    private boolean vert2Init = false;
    @Override
    protected void myCreateScene(int curr) {
        switch (curr){
            case 0:
                arrayVertex = new float[]{
                    0.0f, 0.0f, 0.0f,
                };
                numVertex = 1;
                break;
            case 1:
                if(!vert1Init) {
                    arrVert1 = new float[6 * 8];

```

```

        int pos = 0;
        addRect(
            arrVert1, pos,
            -0.5f, -0.5f, -0.5f,
            0.5f, -0.5f, -0.5f,
            0.5f, -0.5f, 0.5f,
            -0.5f, -0.5f, 0.5f,
            0.01f, 3.99f,
            3.99f, 3.99f,
            3.99f, 0.01f,
            0.01f, 0.01f
        );
        vert1Init = true;
    }
    objColor = new float[]{
        1.0f, 1.0f, 1.0f
    };
    arrayVertex = arrVert1;
    numVertex = 6;
    textureHandle = textureHandle1;
    break;
case 2:
    if(!vert2Init) {
        arrVert2 = new float[6 * 8 * 6];
        int pos = 0;
        addCube(
            arrVert2, pos,
            -0.5f, -0.5f, -0.5f,
            0.5f, -0.5f, -0.5f,
            0.5f, -0.5f, 0.5f,
            -0.5f, -0.5f, 0.5f,
            -0.5f, 0.5f, -0.5f,
            0.5f, 0.5f, -0.5f,
            0.5f, 0.5f, 0.5f,
            -0.5f, 0.5f, 0.5f
        );
        vert2Init = true;
    }
    objColor = new float[]{
        1.0f, 1.0f, 1.0f
    };
    arrayVertex = arrVert2;
    numVertex = 6 * 6;
    textureHandle = textureHandle2;
    break;
default:
    break;
}
}
@Override
public void setObjMat(int currObj){
    if(currObj == 1){
        objMat1 = objMat.clone();
        return;
    }
    if(currObj >= 2 && currObj <= 15){
        robot[currObj-2].partMat = objMat.clone();
    }
}
@Override
public void getObjMat(int currObj){
    if(currObj == 1){
        objMat = objMat1.clone();
        return;
    }
    if(currObj >= 2 && currObj <= 15){
        objMat = robot[currObj-2].partMat.clone();
    }
}
@Override
public boolean myCreateShaderProgram(int curr) {
    switch (curr){
        case 0:
            return myCompileAndAttachShaders(myShaders.vertexShaderCode5,
myShaders.fragmentShaderCode5);
        case 2:

```



```

        return myCompileAndAttachShaders(myShaders.vertexShaderCode4,
myShaders.fragmentShaderCode4);
        default:
            return false;
    }
}
@Override
public void myBindObjBuff(int curr) {
    switch (curr){
        case 0:
            myVertexArrayBind1(arrayVertex, 3,"vPosition", 0);
            break;
        case 2:
            myVertexArrayBind3(arrayVertex, 8,"vPosition", 0,"vNormal", 3*4, "vTexture",
6*4);
            break;
        default:
            break;
    }
}
@Override
public boolean onTouchNotUsed() { return false; }
@Override
public boolean onActionDown(float x, float y, int cx, int cy){
    prevDownX = x;
    prevDownY = y;
    return true;
};
@Override
public boolean onActionMove(float x, float y, int cx, int cy){
    float dx = x - prevDownX;
    float dy = y - prevDownY;
    prevDownX = x;
    prevDownY = y;
    cameraMat[6] -= dx * sensitivity;
    cameraMat[8] += dy * sensitivity;
    float limit = 1.5f;
    if (cameraMat[8] > limit) cameraMat[8] = limit;
    if (cameraMat[8] < -limit) cameraMat[8] = -limit;
    updateCameraOrbitY();
    return true;
};
private void updateCameraOrbitY() {
    float cx = 0.0f;
    float cy = 0.0f;
    float cz = 0.0f;
    float yaw = cameraMat[6];
    float radius = cameraMat[7];
    float pitch = cameraMat[8];
    float x = cx + radius * (float)(cos(pitch) * sin(yaw));
    float y = cy + radius * (float)(sin(pitch));
    float z = cz + radius * (float)(cos(pitch) * cos(yaw));
    cameraMat[0] = x;
    cameraMat[1] = y;
    cameraMat[2] = z;
    cameraMat[3] = cx;
    cameraMat[4] = cy;
    cameraMat[5] = cz;
}
@Override
public boolean onScale(float scaleFactor){
    float centerScaleFactor = 1.0f;
    float moveSensitivity = 0.1f;
    if(centerScaleFactor > scaleFactor){
        cameraMat[7] -= moveSensitivity;
    }
    else {
        cameraMat[7] += moveSensitivity;
    }
    cameraMat[7] = Math.max(1.0f, Math.min(cameraMat[7], 10.0f));
    updateCameraOrbitY();
    return true;
}
@Override
public boolean animation(){
    headAnimation();
}

```

```

        heandRightAnimation();
        heandLeftAnimation();
        legRightAnimation();
        legLeftAnimation();
        robotMove();
        return true;
    }
    private float stepZ = 0.01f;
    private void robotMove(){
        if(robot[1].partMat[2] >= 2.0f){
            stepZ = -0.01f;
        }
        if(robot[1].partMat[2] <= -2.0f){
            stepZ = 0.01f;
        }
        for(int i = 0; i <= 13; i++){
            robot[i].partMat[2] += stepZ;
        }
    }
    private float headRY = 1.5f;
    private void headAnimation(){
        if(robot[0].partMat[6] == 45.0f){
            headRY = -1.5f;
        }
        if(robot[0].partMat[6] == -45.0f){
            headRY = +1.5f;
        }
        robot[0].partMat[6] += headRY;
    }
    private int heandRMode = 2;
    private float heandRrx = 1.0f;
    private void heandRightAnimation(){
        if(heandRMode == 1){
            if(robot[8].partMat[4] == 30.0f){
                heandRrx = -1.0f;
            }
            if(robot[8].partMat[4] == 0.0f){
                heandRMode = 2;
            }
        }
        else{
            if(robot[8].partMat[4] == -30.0f){
                heandRrx = 1.0f;
            }
            if(robot[8].partMat[4] == 0.0f){
                heandRMode = 1;
            }
        }
        float[] topPoint = topPoint(robot[8].partMat[1], robot[8].partMat[2], 0.25f,
robot[8].partMat[4]);
        rotateAroundX(8, topPoint[0], topPoint[1], heandRrx);
        rotateAroundX(9, topPoint[0], topPoint[1], heandRrx);
        rotateAroundX(10, topPoint[0], topPoint[1], heandRrx);
        float[] botPoint = bottomPoint(robot[8].partMat[1], robot[8].partMat[2], 0.25f,
robot[8].partMat[4]);
        float angleK = 1.25f;
        rotateAroundX(9, botPoint[0], botPoint[1], heandRrx * angleK);
        rotateAroundX(10, botPoint[0], botPoint[1], heandRrx * angleK);
    }
    private int heandLMode = 1;
    private float heandLrx = -1.0f;
    private void heandLeftAnimation(){
        if(heandLMode == 1){
            if(robot[11].partMat[4] == 30.0f){
                heandLrx = -1.0f;
            }
            if(robot[11].partMat[4] == 0.0f){
                heandLMode = 2;
            }
        }
        else{
            if(robot[11].partMat[4] == -30.0f){
                heandLrx = 1.0f;
            }
            if(robot[11].partMat[4] == 0.0f){
                heandLMode = 1;
            }
        }
    }

```

```

    }
    float[] topPoint = topPoint(robot[11].partMat[1], robot[11].partMat[2], 0.25f,
robot[11].partMat[4]);
    rotateAroundX(11, topPoint[0], topPoint[1], heandLrx);
    rotateAroundX(12, topPoint[0], topPoint[1], heandLrx);
    rotateAroundX(13, topPoint[0], topPoint[1], heandLrx);
    float[] botPoint = bottomPoint(robot[11].partMat[1], robot[11].partMat[2], 0.25f,
robot[11].partMat[4]);
    float angleK = 1.25f;
    rotateAroundX(12, botPoint[0], botPoint[1], heandLrx * angleK);
    rotateAroundX(13, botPoint[0], botPoint[1], heandLrx * angleK);
}
private int legRMode = 1;
private float legRrx = -1.5f;
private float angleRK = -1.0f;
private void legRightAnimation(){
    if(legRMode == 1){
        if(robot[2].partMat[4] == 45.0f){
            legRrx = -1.5f;
        }
        if(robot[2].partMat[4] == 0.0f){
            legRMode = 2;
            angleRK = -1.0f;
        }
    }
    else{
        if(robot[2].partMat[4] == -45.0f){
            legRrx = 1.5f;
        }
        if(robot[2].partMat[4] == 0.0f){
            legRMode = 1;
            angleRK = 0.5f;
        }
    }
    float[] topPoint = topPoint(robot[2].partMat[1], robot[2].partMat[2], 0.25f,
robot[2].partMat[4]);
    rotateAroundX(2, topPoint[0], topPoint[1], legRrx);
    rotateAroundX(3, topPoint[0], topPoint[1], legRrx);
    rotateAroundX(4, topPoint[0], topPoint[1], legRrx);
    float[] botPoint = bottomPoint(robot[2].partMat[1], robot[2].partMat[2], 0.25f,
robot[2].partMat[4]);
    rotateAroundX(3, botPoint[0], botPoint[1], legRrx * angleRK);
    rotateAroundX(4, botPoint[0], botPoint[1], legRrx * angleRK);
}
private int legLMode = 2;
private float legLrx = 1.5f;
private float angleLK = 0.5f;
private void legLeftAnimation(){
    if(legLMode == 1){
        if(robot[5].partMat[4] == 45.0f){
            legLrx = -1.5f;
        }
        if(robot[5].partMat[4] == 0.0f){
            legLMode = 2;
            angleLK = -1.0f;
        }
    }
    else{
        if(robot[5].partMat[4] == -45.0f){
            legLrx = 1.5f;
        }
        if(robot[5].partMat[4] == 0.0f){
            legLMode = 1;
            angleLK = 0.5f;
        }
    }
    float[] topPoint = topPoint(robot[5].partMat[1], robot[5].partMat[2], 0.25f,
robot[5].partMat[4]);
    rotateAroundX(5, topPoint[0], topPoint[1], legLrx);
    rotateAroundX(6, topPoint[0], topPoint[1], legLrx);
    rotateAroundX(7, topPoint[0], topPoint[1], legLrx);
    float[] botPoint = bottomPoint(robot[5].partMat[1], robot[5].partMat[2], 0.25f,
robot[5].partMat[4]);
    rotateAroundX(6, botPoint[0], botPoint[1], legLrx * angleLK);
    rotateAroundX(7, botPoint[0], botPoint[1], legLrx * angleLK);
}

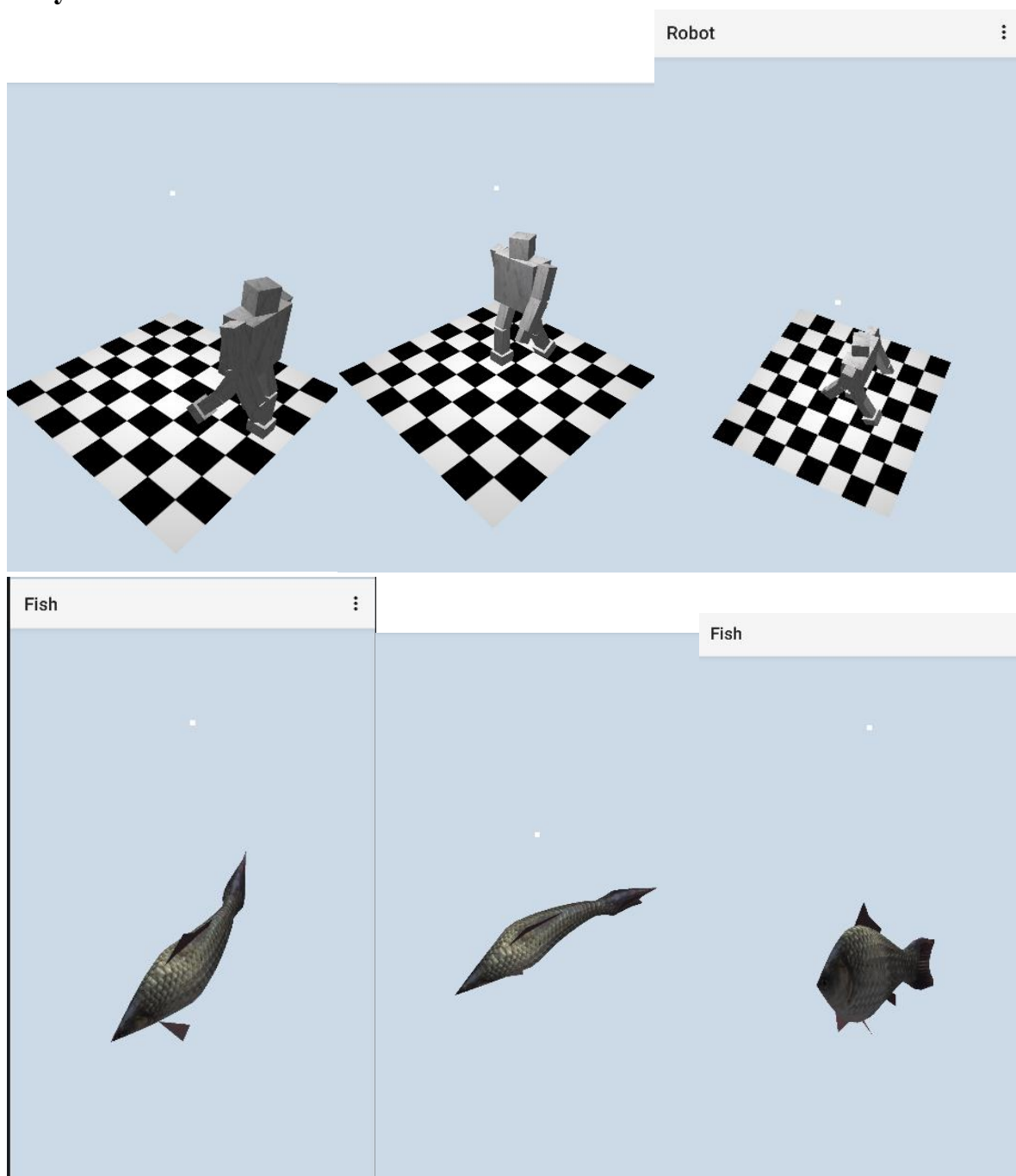
```

```

    }
    private float[] topPoint(float cy, float cz, float r, float rx) {
        float theta = (float)(Math.toRadians(rx + 0));
        float y = cy + r * (float)(Math.cos(theta));
        float z = cz + r * (float)(Math.sin(theta));
        return new float[]{y, z};
    }
    public static float[] bottomPoint(float cy, float cz, float r, float rx) {
        float theta = (float)(Math.toRadians(rx - 180));
        float y = cy + r * (float)(Math.cos(theta));
        float z = cz + r * (float)(Math.sin(theta));
        return new float[]{y, z};
    }
    private void rotateAroundX(int index, float cY, float cZ, float angleStep){
        float angle = (float)Math.toRadians(angleStep);
        float y = robot[index].partMat[1];
        float z = robot[index].partMat[2];
        float dy = y - cY;
        float dz = z - cZ;
        float newY = dy * (float)Math.cos(angle) - dz * (float)Math.sin(angle);
        float newZ = dy * (float)Math.sin(angle) + dz * (float)Math.cos(angle);
        robot[index].partMat[1] = newY + cY;
        robot[index].partMat[2] = newZ + cZ;
        robot[index].partMat[4] += angleStep;
    }
}

```

Результати:



Висновок

У ході виконання лабораторної роботи №6 я отримав практичні навички створення та анімації рухомих об'єктів у тривимірних сценах за допомогою OpenGL ES. Я навчився формувати складні об'єкти з простих примітивів, застосовувати модельні перетворення (зсув, обертання) та реалізовувати рух частин об'єкта на основі кінематичних залежностей .

Також я ознайомився з принципами скелетної анімації та роботою з матрицями трансформацій і шейдерами. У результаті я закріпив навички створення динамічних 3D-сцен та отримав базу для подальшої роботи з комп'ютерною графікою